In [1]:
```
# 1. Import necessary packages
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf

from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Conv1D, MaxPooling1D, UpSampling1D
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, classification_report

# 2. Load ECG dataset
data = pd.read_csv('ecg.csv')

print(data.head())
print(data.columns)

# 3. Separate features and labels
# Assumption: Last column contains labels
X = data.iloc[:, :-1].values
y = data.iloc[:, -1].values

print("Label values:", np.unique(y))

# 0 = Normal, 1 = Abnormal
normal_label = 0

# 4. Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Select only normal ECG signals for training
X_normal = X_train[y_train == normal_label]

# Split normal data into training and validation
X_train_normal, X_val_normal = train_test_split(
    X_normal, test_size=0.1, random_state=42
)

# 5. Preprocess ECG signals
# Make signal length divisible by 4
length = (X_train_normal.shape[1] // 4) * 4

X_train_normal = X_train_normal[:, :length]
X_val_normal = X_val_normal[:, :length]
X_test = X_test[:, :length]

# Scale the data
scaler = StandardScaler()

X_train_normal = scaler.fit_transform(X_train_normal)
X_val_normal = scaler.transform(X_val_normal)
X_test = scaler.transform(X_test)

# Reshape for Conv1D
X_train_normal = X_train_normal.reshape(-1, length, 1)
```

```
X_val_normal = X_val_normal.reshape(-1, length, 1)
X_test = X_test.reshape(-1, length, 1)

# 6. Build Convolutional Autoencoder

input_layer = Input(shape=(length, 1))

# Encoder
x = Conv1D(16, 3, activation='relu', padding='same')(input_layer)
x = MaxPooling1D(2, padding='same')(x)

x = Conv1D(8, 3, activation='relu', padding='same')(x)
encoded = MaxPooling1D(2, padding='same')(x)

# Decoder
x = Conv1D(8, 3, activation='relu', padding='same')(encoded)
x = UpSampling1D(2)(x)

x = Conv1D(16, 3, activation='relu', padding='same')(x)
x = UpSampling1D(2)(x)

decoded = Conv1D(1, 3, activation='linear', padding='same')(x)

autoencoder = Model(input_layer, decoded)

# 7. Compile the model
autoencoder.compile(
    optimizer='adam',
    loss='mse'
)

autoencoder.summary()

# 8. Train using only normal ECG signals
history = autoencoder.fit(
    X_train_normal,
    X_train_normal,
    epochs=10,
    batch_size=32,
    validation_data=(X_val_normal, X_val_normal)
)

# 9. Calculate reconstruction error on normal validation data
val_predictions = autoencoder.predict(X_val_normal, verbose=0)

val_error = np.mean(
    np.square(X_val_normal - val_predictions),
    axis=(1, 2)
)

# Set anomaly threshold
threshold = np.percentile(val_error, 95)

print("Anomaly Threshold:", threshold)

# 10. Predict test ECG signals
test_predictions = autoencoder.predict(X_test, verbose=0)

test_error = np.mean(
    np.square(X_test - test_predictions),
```

```
    axis=(1, 2)
)

# Detect anomalies
y_pred = (test_error > threshold).astype(int)

# 11. Evaluate the model
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# 12. Plot training loss
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.title('ECG Autoencoder Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()

# 13. Display anomaly detection results
print("Total ECG Signals:", len(y_test))
print("Actual Abnormal Signals:", np.sum(y_test != normal_label))
print("Detected Abnormal Signals:", np.sum(y_pred))

# 14. Visualize original and reconstructed ECG signal
plt.figure(figsize=(10, 4))

plt.plot(X_test[0].flatten(), label='Original ECG')
plt.plot(test_predictions[0].flatten(), label='Reconstructed ECG')

plt.title('ECG Signal Reconstruction')
plt.xlabel('Time Steps')
plt.ylabel('Signal Value')
plt.legend()
plt.show()
```

```
   -0.11252183  -2.8272038  -3.7738969  -4.3497511  -4.376041  -3.4749863  \
0    -1.100878   -3.996840   -4.285843   -4.506579  -4.022377   -3.234368
1    -0.567088   -2.593450   -3.874230   -4.584095  -4.187449   -3.151462
2     0.490473   -1.914407   -3.616364   -4.318823  -4.268016   -3.881110
3     0.800232   -0.874252   -2.384761   -3.973292  -4.338224   -3.802422
4    -1.507674   -3.574550   -4.478011   -4.408275  -3.321242   -2.105171

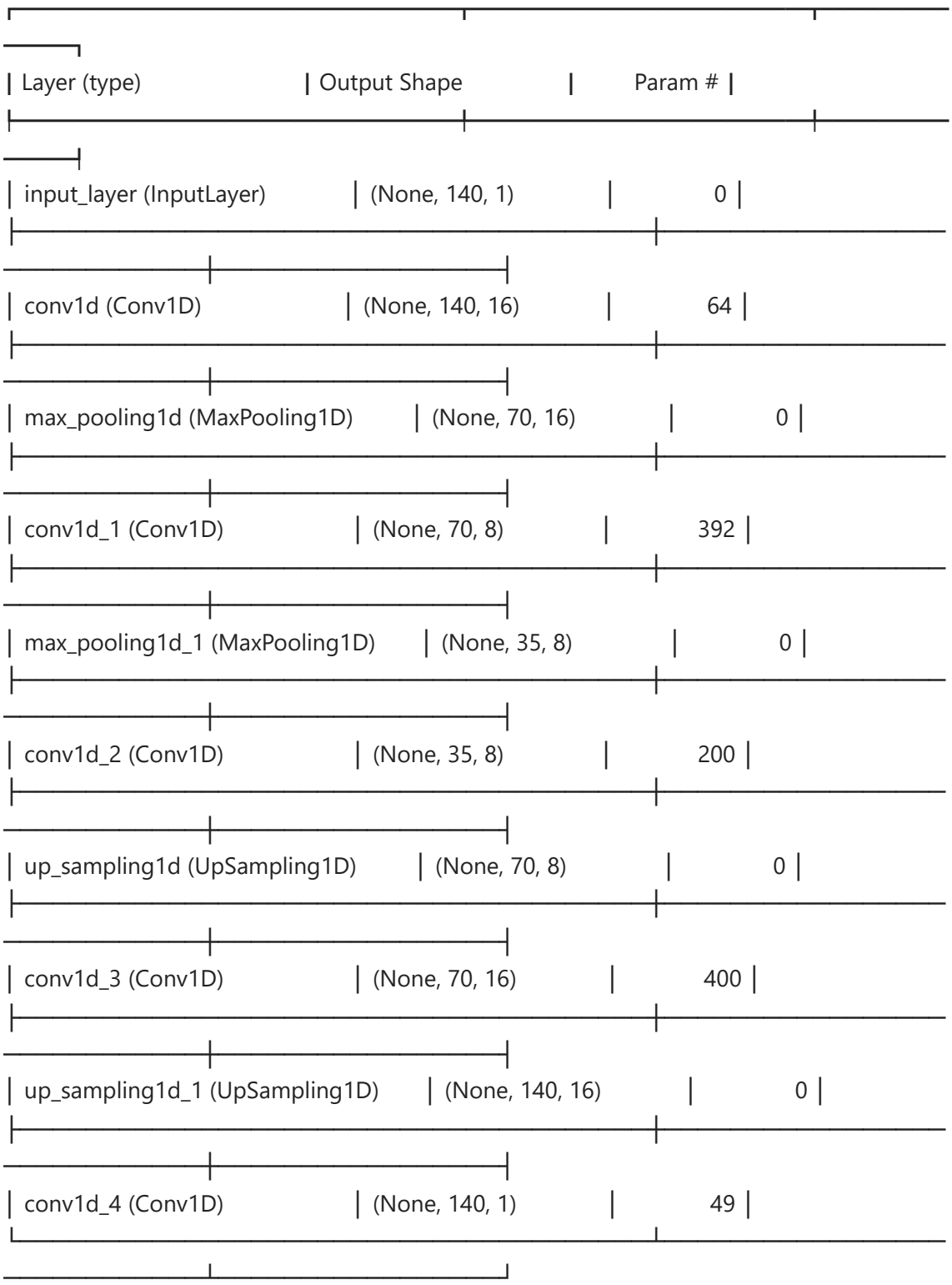
   -2.1814082  -1.8182865  -1.2505219  -0.47749208  ...  0.79216787  \
0   -1.566126   -0.992258   -0.754680     0.042321  ...    0.538356
1   -1.742940   -1.490659   -1.183580    -0.394229  ...    0.886073
2   -2.993280   -1.671131   -1.333884    -0.965629  ...    0.350816
3   -2.534510   -1.783423   -1.594450    -0.753199  ...    1.148884
4   -1.481048   -1.301362   -0.498240    -0.286928  ...    1.089068

   0.93354122  0.79695779  0.57862066  0.2577399  0.22807718  0.12343082  \
0    0.656881    0.787490    0.724046   0.555784    0.476333    0.773820
1    0.531452    0.311377   -0.021919  -0.713683   -0.532197    0.321097
2    0.499111    0.600345    0.842069   0.952074    0.990133    1.086798
3    0.958434    1.059025    1.371682   1.277392    0.960304    0.971020
4    0.983369    1.014124    0.952629   0.749326    1.007076    1.634990

   0.92528624  0.19313742  1.0
0    1.119621   -1.436250  1.0
1    0.904227   -0.421797  1.0
2    1.403011   -0.383564  1.0
3    1.614392    1.421456  1.0
4    1.493365   -0.783134  1.0

[5 rows x 141 columns]
Index(['-0.11252183', '-2.8272038', '-3.7738969', '-4.3497511', '-4.376041',
       '-3.4749863', '-2.1814082', '-1.8182865', '-1.2505219', '-0.47749208',
       ...
       '0.79216787', '0.93354122', '0.79695779', '0.57862066', '0.2577399',
       '0.22807718', '0.12343082', '0.92528624', '0.19313742', '1.0'],
      dtype='str', length=141)
Label values: [0. 1.]
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
```

Model: "functional"



Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 140, 1)	0
conv1d (Conv1D)	(None, 140, 16)	64
max_pooling1d (MaxPooling1D)	(None, 70, 16)	0
conv1d_1 (Conv1D)	(None, 70, 8)	392
max_pooling1d_1 (MaxPooling1D)	(None, 35, 8)	0
conv1d_2 (Conv1D)	(None, 35, 8)	200
up_sampling1d (UpSampling1D)	(None, 70, 8)	0
conv1d_3 (Conv1D)	(None, 70, 16)	400
up_sampling1d_1 (UpSampling1D)	(None, 140, 16)	0
conv1d_4 (Conv1D)	(None, 140, 1)	49

Total params: 1,105 (4.32 KB)

Trainable params: 1,105 (4.32 KB)

Non-trainable params: 0 (0.00 B)

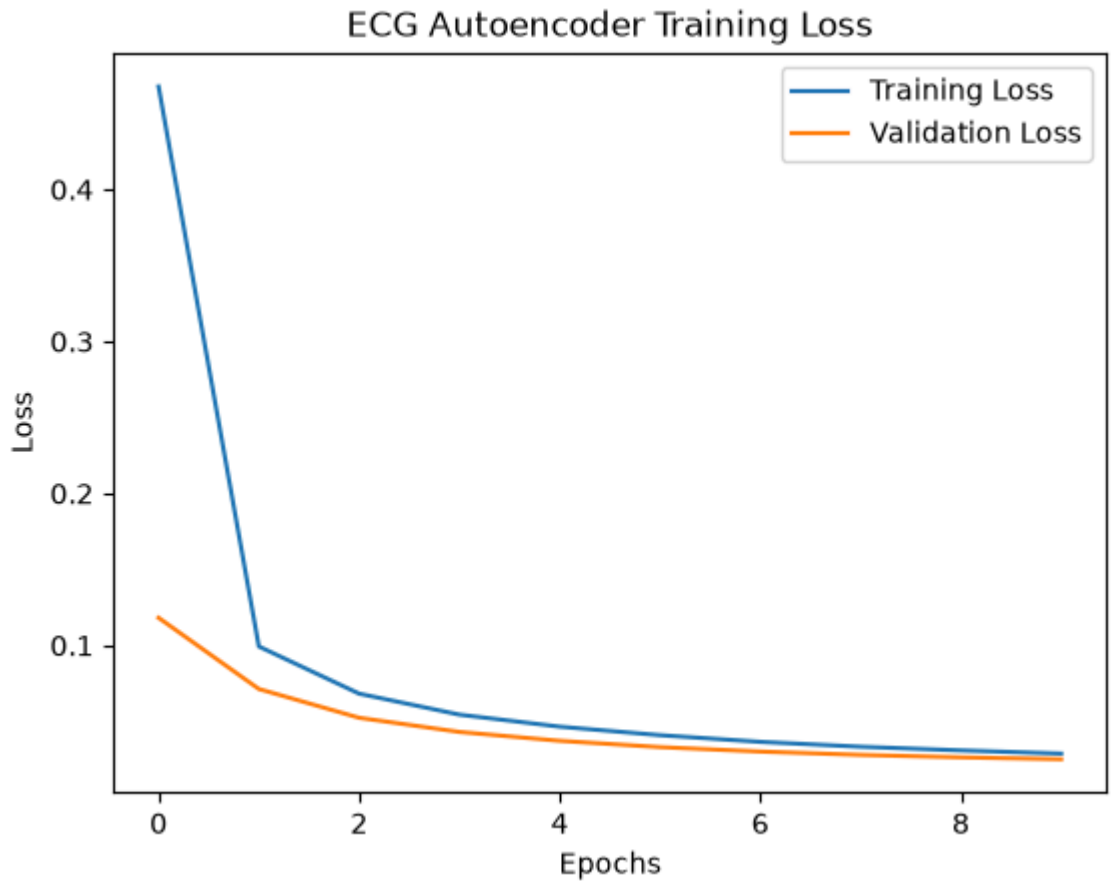
```
Epoch 1/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 4s 19ms/step - loss: 0.4678 - val_loss: 0.1181
Epoch 2/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0993 - val_loss: 0.0713
Epoch 3/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0682 - val_loss: 0.0523
Epoch 4/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.0544 - val_loss: 0.0430
Epoch 5/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.0465 - val_loss: 0.0371
Epoch 6/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.0409 - val_loss: 0.0330
Epoch 7/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - loss: 0.0365 - val_loss: 0.0301
Epoch 8/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.0333 - val_loss: 0.0279
Epoch 9/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.0309 - val_loss: 0.0263
Epoch 10/10
47/47 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.0287 - val_loss: 0.0250
Anomaly Threshold: 0.04761839092522644

Confusion Matrix:
[[387  29]
 [198 386]]

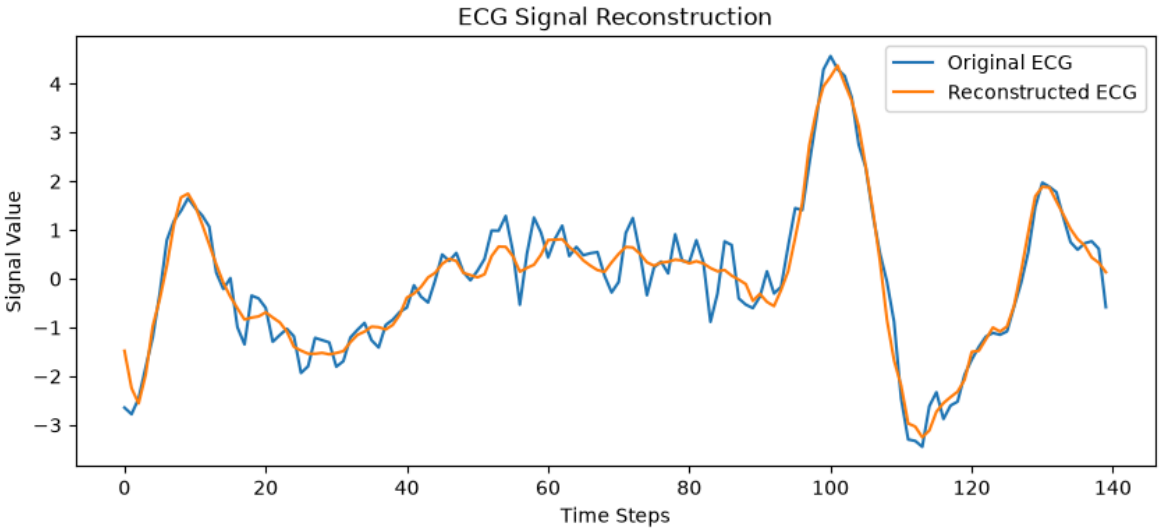
Classification Report:
              precision    recall  f1-score   support

         0.0       0.66      0.93      0.77       416
         1.0       0.93      0.66      0.77       584

    accuracy                           0.77      1000
   macro avg       0.80      0.80      0.77      1000
weighted avg       0.82      0.77      0.77      1000
```




```
Total ECG Signals: 1000
Actual Abnormal Signals: 584
Detected Abnormal Signals: 415
```



In [ ]: